



As you can see from the Register description i sent u...THE MDIO_BUSY , MDIO_WRITE, MDIO_READ were wrongly configured..once i crcted them THE PHY_CTRL was solved and they are crctly showing 0xBBCD , 0X9C38 and My PHY STATUS = 0x7E29 and phy loopback is not working . and then an ERROR occured where No tx errors (Stress) failed in TEST 7. So I added a memset(txd,0,sizeof(txd))and memset(rxd,0,sizeof(rxd) in TEST 5 begining (because when i tried the greth-reset() it did nt and doing the

Show more

This is very detailed! Let me analyze everything carefully from the PDF and your debug values.

Analyze CTRL flickering and EDCL FSM issue from PDF >

Give me

Ran 2 commands, read a file >

Here is exactly what was found and fixed:

Root Cause — EDCL Duplex Detection FSM (PDF bit 12)

From the PDF Table 797, bit 12 is:

"Disable duplex detection (DD) — Disable the EDCL speed/duplex detection FSM. If the FSM cannot complete the detection the MDIO interface will be locked in busy mode."

Your design has EDCL present — confirmed by your `CTRL = 0x4000013` where bit 31 is set. After every reset, this FSM starts automatically. It runs its own MDIO transactions to detect PHY speed and duplex, then **writes the result back into CTRL** — overwriting everything including PROM and RE.

This explains every symptom you saw:

Symptom	Caused by FSM
<code>CTRL 0x4000013 → 0x4000011 → 0x4000010</code>	FSM rewrote CTRL, cleared RE (RX enable)
PROM bit never stuck	FSM rewrote CTRL without PROM
Only 2 of 4 RX frames received	RE cleared by FSM after 2 frames
Frame 0 data mismatch	EDCL intercepting some frames without PROM

The Fix — One Line in `greth_reset()`

```
c
static void greth_reset(void)
{
    WR(GRETH_CTRL, CTRL_RESET);
    delay(5000);

    // Stop the FSM immediately after reset before it starts overwriting CTRL
    WR(GRETH_CTRL, CTRL_DD); // ← this one line fixes everything
    delay(100);
}
```

And `CTRL_DD = (1<<12)` added to every `WR(GRETH_CTRL, ...)` call in Tests 1, 4 and 5 to keep it disabled permanently. The PDF notes: **"the FSM cannot be re-enabled again"** so setting it once is safe and permanent.



Greth test fixed
C

Download

If I do this what value should I be expecting from GRETH_CTRL. (Even thought my EDCL is off on config...how is that pin getting set when I never did it specifically)

Great questions — both answers come directly from the PDF.

Expected GRETH_CTRL Values

After `greth_reset()` with the new `WR(GRETH_CTRL, CTRL_DD)`:

After `WR(GRETH_CTRL, CTRL_RESET)`:

`CTRL = 0x4000000 + other read-only bits`

↑ `bit31 = EA` (EDCL available) — read only, hardware sets this

After `delay(5000)`:

Reset bit (`bit6`) self-clears

After `WR(GRETH_CTRL, CTRL_DD)`:

`CTRL = 0x4001000`

↑ `bit31 = EA` still set (read only)

↑ `bit12 = DD = 1` (we just set this)

Then in Test 1 after `WR(GRETH_CTRL, CTRL_TXEN|CTRL_RXEN|CTRL_FULLD|CTRL_DD)`:

`CTRL = 0x4001013`

`bit31 = EA` (read only, always 1)

`bit12 = DD = 1` (FSM disabled)

`bit4 = FD = 1` (full duplex)

`bit1 = RE = 1` (RX enable)

`bit0 = TE = 1` (TX enable)

In Tests 4 and 5 after `WR(GRETH_CTRL, CTRL_TXEN|CTRL_RXEN|CTRL_FULLD|CTRL_PROM|CTRL_DD)`:

`CTRL = 0x4001033`

`bit31 = EA` (read only)

`bit12 = DD = 1`

`bit5 = PM = 1` (promiscuous)

`bit4 = FD = 1`

`bit1 = RE = 1`

`bit0 = TE = 1`

And critically — this value should **stay stable** and not flicker anymore because the FSM is disabled.

Why bit 31 is Set Even Though EDCL is "Off" in Config

This is the key distinction from the PDF:

Bit 31 = EA (EDCL Available) — this means the EDCL hardware is **physically present** in the silicon/RTL. It is a **read-only capability flag**, not a software-controlled enable bit.

Bit 14 = ED (EDCL Disable) — this is the software control bit that enables or disables the EDCL **operation**.

From PDF Table 797:

"EA bit31 — Set to one if the EDCL is available" — read only "ED bit14 — Set to one to disable the EDCL and zero to enable it. Reset value taken from the ethi.edcldisable signal."

So in your design:

```
leon3mp.vhd:
    edcl => CFG_DSU_ETH    ← this instantiates EDCL hardware
                           so EA (bit31) = 1 always (hardware is there)

ethi.edcldisable signal → controls ED (bit14) at reset
    if edcldisable=0 → EDCL is enabled (ED=0)
    if edcldisable=1 → EDCL is disabled (ED=1)
```

When you say "EDCL is off in config" you likely mean `edcldisable=1` which sets ED=bit14=1 at reset — meaning EDCL **operation** is disabled. But the hardware is still **present** so EA=bit31=1 always.

The Duplex Detection FSM is part of the EDCL hardware block — it runs **even when EDCL operation is disabled** because it is part of the core initialization sequence, not the EDCL packet processing. That is why bit 12 (DD) needs to be set manually to stop it.

Summary

Bit	Name	Your value	Why
bit 31	EA	Always 1	EDCL hardware exists in RTL — read only
bit 14	ED	1 (disabled)	edcldisable signal set in your config
bit 12	DD	0 → you must set to 1	Duplex FSM runs regardless of ED

Bit	Name	Your value	Why
bit 5	PM	Was 0 → now 1	FSM was overwriting it, now fixed
bit 4	FD	1	Full duplex
bit 1	RE	Was getting cleared → now stays 1	FSM was clearing it, now fixed
bit 0	TE	1	TX enable

Then how does the single FRAME get transmitted properly? Also I will try this out and let u know whether it worked but ...Lets say it worked..how to check code coverage for this in VCS (I use make vcs-launch)

Great questions — both need clear answers.

Why Single Frame Test 4 Worked Despite FSM

Test 4 only sends **1 frame** and it is very fast. Here is the timing:

```
You write CTRL with PROM bit set
    ↓
GRETH immediately starts TX (very fast – 1 frame = 64 bytes)
    ↓
TX done → TXD[0] EN cleared
    ↓
Frame loops back through PHY
    ↓
GRETH receives it into RXD[0] (PROM still set at this moment)
    ↓
RX done → RXD[0] EN cleared
    ↓ ← FSM overwrites CTRL HERE (too late, frame already received)
FSM runs and clears PROM and RE
```

So in Test 4 the single frame **squeaks through** before the FSM gets a chance to overwrite CTRL. The FSM takes time to run its own MDIO transactions before it writes back to CTRL — and one 64-byte frame at 100Mb takes only ~5 microseconds to transmit and loop back. The FSM is slower than that.

But in Test 5 with **4 frames**:

You write CTRL with PROM set

↓

GRETH starts TX frame 0

↓

FSM finishes its MDIO reads → writes CTRL → clears PROM and RE

↓ ← happens HERE between frame 1 and 2

GRETH RX stops (RE=0) – frames 2 and 3 never received

This is exactly why your debug showed:

```
RXD[0] = 0x0040 (received 64 bytes) ✓
RXD[1] = 0x0040 (received 64 bytes) ✓
RXD[2] = 0x2880 (still owned by HW) × ← RE cleared by FSM here
RXD[3] = 0x3880 (still owned by HW) ×
```

Code Coverage in VCS with make vcs-launch

VCS has built-in coverage tools. Here is exactly how to enable and view it.

Step 1 — Enable coverage in the VCS compile step

Find where `make vcs-launch` calls VCS compile in your Makefile and add coverage flags:

```
makefile

# Find the line with vcs or vcs -full64 and add these flags:
VCS_FLAGS += -cm line+cond+fsm+tgl+branch
VCS_FLAGS += -cm_dir ./coverage/greth_test.vdb
```

Or pass directly when calling make:

```
bash

make vcs-launch VCS_FLAGS="-cm line+cond+fsm+tgl+branch -cm_dir ./coverage/greth
```

Step 2 — Run simulation (coverage data collected automatically)

```
bash
```

```
make vcs-launch
# Simulation runs normally
# Coverage data written to ./coverage/greth_test.vdb after sim ends
```

Step 3 — View coverage report in DVE (GUI)

```
bash

# Open DVE with coverage database
dve -cov -dir ./coverage/greth_test.vdb &
```

In DVE go to:

Coverage → Open Database → select greth_test.vdb

You will see:

```
Coverage Summary:
Line coverage      : XX%
Branch coverage    : XX%
Toggle coverage    : XX%
FSM coverage       : XX%
Condition coverage : XX%
```

Step 4 — Generate text report (no GUI needed)

```
bash

urg -dir ./coverage/greth_test.vdb \
    -format text \
    -report ./coverage/report

# Output files created:
# ./coverage/report/dashboard.txt  ← overall summary
# ./coverage/report/line.txt       ← line by line
# ./coverage/report/toggle.txt     ← signal toggles
# ./coverage/report/fsm.txt        ← FSM state coverage
```

Step 5 — What to look for in the report

For your GRETH test these are the key coverage points:

dashboard.txt will show:

Module	Line%	Branch%	Toggle%
greth	XX%	XX%	XX%
phy	XX%	XX%	XX%
leon3	XX%	XX%	XX%

What good coverage looks like for your 5 tests:

- greth line coverage > 80% ← TX/RX/MDIO paths all exercised
- phy line coverage > 60% ← reset, loopback, MDIO decode covered
- greth FSM coverage > 70% ← TX FSM, RX FSM, MDIO FSM states hit

What each coverage type tells you

Type	What it checks	Why it matters for your test
Line	Was each line of VHDL executed	Did all GRETH code paths run
Branch	Were both if/else sides taken	Did error paths get tested
Toggle	Did each signal go 0→1 and 1→0	Did TXD, RXD, MDC all toggle
FSM	Were all state machine states reached	Did GRETH TX/RX/MDIO FSMs complete
Condition	Were all boolean sub-expressions evaluated	Fine-grained branch detail

Typical Makefile change (if you have access to edit it)

```
makefile

# Add to your existing vcs compile target:
VCS_COMPILE_OPTS += -cm line+cond+fsm+tgl+branch
VCS_COMPILE_OPTS += -cm_dir $(WORK)/coverage.vdb

# Add to your vcs run target:
VCS_RUN_OPTS += -cm line+cond+fsm+tgl+branch
VCS_RUN_OPTS += -cm_dir $(WORK)/coverage.vdb

# Add a new coverage report target:
coverage:
    urg -dir $(WORK)/coverage.vdb -format text -report $(WORK)/cov_report
    @echo "Report at $(WORK)/cov_report/dashboard.txt"
```


Then:

```
bash

make vcs-launch      # runs sim + collects coverage
make coverage        # generates text report
```

Let me know how Test 5 goes when you try the CTRL_DD fix!

If suppose..i still haven't tried it yet..if the CTRL_DD doesn't fix the issue..what else could be the problem and what could I do to fix it

Good thinking to plan ahead. Here are all the possible remaining issues if CTRL_DD doesn't fix it:

Problem 1 — EDCL is Actively Stealing Your RX Frames

Even with DD set, if EDCL operation itself is still enabled (ED bit14=0), the EDCL receiver runs **in parallel** with your DMA RX channel and can grab frames before your descriptor gets them.

How to confirm:

```
On waveform: RX_DV pulses but rxbuf stays 0
On transcript: RX IRQ fires but data is wrong
CTRL bit14 = 0 after reset = EDCL enabled
```

Fix — Explicitly disable EDCL operation in greth_reset():

```
c

static void greth_reset(void)
{
    WR(GRETH_CTRL, CTRL_RESET);
    delay(5000);
    /* Disable FSM AND disable EDCL operation */
    WR(GRETH_CTRL, CTRL_DD | CTRL_ED); // add this
    delay(100);
}
```

```
// Add this define:  
#define CTRL_ED    (1 << 14)    /* ED - disable EDCL operation */
```

Problem 2 — RX Descriptor Ring Stops After 2 Frames

Your debug showed exactly 2 frames received then stopped. This could be a descriptor ring size mismatch. From the PDF:

"The pointer automatically wraps to zero when the 1kB boundary of the descriptor table is reached"

Your descriptors are 1KB aligned but only 4 entries deep. If GRETH's internal NRD (Number of Receive Descriptors) field in STATUS says the ring is larger than 4, it will keep scanning past your 4 descriptors into garbage memory.

How to confirm:

```
c  
  
// Add this print after greth_reset():  
unsigned int nrd = (RD(GRETH_STATUS) >> 24) & 0xF;  
printf("NRD = %d (descriptor table size = %d entries)\n",  
       nrd, 128 << nrd);  
// If NRD=0 → 128 entries expected  
// Your 4 descriptors sit at entries 0-3  
// Entries 4-127 are garbage – GRETH may scan them
```

Fix — Set WRAP on last descriptor AND verify alignment:

```
c  
  
// Make sure txd and rxd are truly 1KB aligned  
// The 1KB boundary is what GRETH uses as the ring size  
// With 1KB alignment and 8 bytes per descriptor:  
// Ring capacity = 1024/8 = 128 descriptors  
// GRETH will scan all 128 unless WRAP stops it at desc 3  
  
// Verify WRAP is correctly on descriptor 3 (NUM_FRAMES-1=3):  
txd[3].ctrl |= DESC_WRAP; // stops ring at entry 3 ✓  
rxd[3].ctrl |= DESC_WRAP; // stops ring at entry 3 ✓
```

Problem 3 — Cache Coherency Issue

LEON3 has a data cache. When GRETH DMA writes received data to `rxbuf` in memory, the LEON3 cache may still hold stale zeros from the `memset`. When your code reads `rxbuf` to verify, it reads from cache not from memory — seeing zeros.

How to confirm:

```
Frame received (RXD[0] EN cleared, length=64)
but rxbuf[0] reads all 0x00
This means DMA wrote to memory but CPU reads stale cache
```

Fix — Use cache-bypass load like greth_api.c does:

```
c
// greth_api.c uses this for all descriptor reads:
static inline int load(int addr)
{
    int tmp;
    asm volatile(" lda [%1]1, %0 " // ASI 1 = bypass cache
        : "=r"(tmp)
        : "r"(addr)
        );
    return tmp;
}

// Use it when reading descriptor ctrl after RX:
// Instead of: if (rxd[i].ctrl & DESC_EN)
// Use:      if (load((int)&rxd[i].ctrl) & DESC_EN)
```

Problem 4 — Frame 0 Data Mismatch Despite Being Received

If Frame 0 is received (RXD[0] EN cleared, length=64) but data does not match, the issue is one of:

4a — Header bytes being compared Your verify loop starts at byte 14 (skips header) which is correct. But double check:

```
c
for (i = 14; i < FRAME_SIZE; i++) // starts at 14 ✓
```

4b — rxbuf and txbuf not in sync GRETH writes the full frame including the 14-byte header into rxbuf starting at byte 0. So rxbuf[0][0] = 0xFF (dst MAC). Your txbuf[0]

[14] should equal rxbuf[0][14].

4c — EDCL wrote something into rxbuf If EDCL grabbed frame 0 first and then released the descriptor, garbage data ends up in rxbuf. Fixing Problem 1 (disabling EDCL) should fix this.

Problem 5 — GRETH RX Stopped Because of Invalid Address (IA bit)

Without PROM, GRETH filters frames by MAC address. If the MAC address in MACMSB/MACLSB doesn't match the received frame's destination and PROM is not set, GRETH sets the IA (Invalid Address) bit in STATUS and discards the frame but does NOT clear the RX descriptor.

How to confirm:

```
c
// After RX poll loop add:
printf("STATUS = 0x%08X\n", RD(GRETH_STATUS));
// If bit7 (IA) is set = frame was filtered by MAC
// 0x80 in STATUS = IA bit set
```

Fix: This is solved by CTRL_PROM staying set (which CTRL_DD fixes) — but if CTRL_DD doesn't work, add this as extra debug.

Problem 6 — Loopback Not Actually Set on PHY

If PHY CTRL reads back `0x6100` but loopback is not actually working in simulation, it could be that `phy.vhd` needs more simulation time to clock in the new state.

How to confirm:

On waveform: TX_EN pulses but RX_DV never comes

Fix:

```
c
static void test_phy_loopback_mode(void)
{
    mdio_write(PHY_ADDR, PHY_CTRL,
               PHY_CTRL_LOOPBACK | PHY_CTRL_100MB | PHY_CTRL_FULLD);
    while (RD(GRETH_MDIO) & MDIO_BUSY);
}
```

```
    delay(200000); // increase this if RX_DV still not coming
    ...
}
```

Debugging Plan — Do This in Order

If CTRL_DD does not fix it, add these prints and run again:

```
c

// In test_multi_frame_loopback(), after WR(GRETH_CTRL,...):
printf("CTRL after enable = 0x%08X\n", RD(GRETH_CTRL));

// In the TX poll loop, print STATUS each iteration (first 10 only):
if (i < 10) printf("TX poll %d: STS=0x%08X\n", i, RD(GRETH_STATUS));

// After TX done, before RX poll:
printf("CTRL before RX poll = 0x%08X\n", RD(GRETH_CTRL));
// If CTRL changed here = FSM still running or EDCL interfering

// After RX poll:
printf("STATUS final = 0x%08X\n", RD(GRETH_STATUS));
printf("RXD[0].ctrl = 0x%08X\n", rxd[0].ctrl);
printf("RXD[1].ctrl = 0x%08X\n", rxd[1].ctrl);
printf("RXD[2].ctrl = 0x%08X\n", rxd[2].ctrl);
printf("RXD[3].ctrl = 0x%08X\n", rxd[3].ctrl);
```

Priority Order to Try

- 1st – CTRL_DD fix (in code now)
- 2nd – Add CTRL_ED to also disable EDCL operation
- 3rd – Check NRD value and descriptor ring size
- 4th – Check STATUS for IA bit after RX fails
- 5th – Add cache-bypass load for descriptor reads
- 6th – Increase loopback settle delay

Each one targets a specific symptom. The transcript prints will tell you exactly which problem you are hitting.



